



مریم رضانی

طراحی پایگاه داده‌ها

بهار ۱۴۰۵



جست وجوی

تمام متن در

مقالات علمی

پروژه: جست‌وجوی مقالات علمی

معرفی دیتاست

در این پروژه هدف ما کار با بخشی از دیتاست مقالات علمی arXiv است. این دیتاست شامل اطلاعات متادیتای مقالات مانند عنوان، چکیده، نویسندگان، دسته‌بندی علمی و تاریخ به‌روزرسانی است. زیرمجموعه مورد استفاده در **این لینک** قرار دارد و باید از آن استفاده نمایید.

فرمت کلی هر رکورد به شکل زیر است:

```
{
  "id": "2101.00001",
  "title": "Efficient Query Processing in Database Systems",
  "abstract": "In this paper, we study ...",
  "authors": "A. Smith, B. Lee",
  "categories": "cs.DB cs.IR",
  "update_date": "2021-01-03"
}
```

در این پروژه مهم‌ترین فیلدهای مورد استفاده، title و abstract هستند. با این حال، از فیلدهایی مانند categories و update_date نیز برای فیلتر، تحلیل و بهینه‌سازی استفاده خواهد شد.

تکنولوژی‌های مناسب برای ذخیره‌سازی و کار با دیتاست

از آنجا که عمده کارهای این پروژه حول جست‌وجو روی داده‌های متنی است، استفاده از تکنولوژی‌هایی مانند Elasticsearch یا OpenSearch مناسب‌تر است. با این حال، شما محدود به این دو تکنولوژی نیستید و می‌توانید با ارائه دلیل کافی از تکنولوژی پیشنهادی خود استفاده کنید.

انتخاب تکنولوژی باید در گزارش توجیه شود. این توجیه حتی در صورت انتخاب Elasticsearch نیز لازم است. در گزارش باید توضیح دهید چرا تکنولوژی انتخاب شده برای این نوع داده و نیازمندی‌های پروژه مناسب است و چه مزایا و معایبی نسبت به حداقل یک گزینه دیگر دارد.

پیش‌پردازش و ذخیره‌سازی دیتاست

در ابتدا لازم است دیتاست داده‌شده را بررسی و پیش‌پردازش کنید. مرحله پیش‌پردازش باید حتماً با زبان Python پیاده‌سازی شود. استفاده از کتابخانه‌های کمکی مجاز است، اما منطق اصلی پاک‌سازی و تبدیل داده باید در کد شما مشخص و قابل بررسی باشد.

حداقل عملیات مورد انتظار در این بخش عبارت‌اند از:

- حذف رکوردهایی که title یا abstract نامعتبر یا خالی دارند.
- پاک‌سازی متن از فاصله‌ها و کاراکترهای غیرضروری.
- تبدیل categories به قالب مناسب برای جست‌وجو و فیلتر.
- استخراج سال از update_date.
- ساخت یک فیلد ترکیبی از title و abstract برای جست‌وجوی کلی.

سپس لازم است ساختار مناسبی برای ذخیره‌سازی داده‌ها در تکنولوژی انتخابی خود طراحی کنید. برای مثال، در صورت استفاده از Elasticsearch یا OpenSearch باید یک mapping مناسب برای فیلدها تعریف کنید و توضیح دهید هر فیلد با چه نوعی ذخیره شده است.

پس از آن، برنامه‌ای توسعه دهید که داده‌های پیش‌پردازش شده را در پایگاه داده ذخیره کند. ورود داده به پایگاه داده باید به صورت دسته‌ای انجام شود. بنابراین استفاده از روش‌هایی مانند bulk insert یا روش معادل آن در تکنولوژی انتخابی الزامی است و ورود تک‌به‌تک رکوردها بدون استفاده از روش دسته‌ای قابل قبول نیست.

متریک‌های مورد انتظار

اکثر دیتابیس‌ها و تکنولوژی‌های ذخیره‌سازی داده، متریک‌هایی از وضعیت خود و کوئری‌های اجرا شده ارائه می‌دهند. شما باید حداقل ۵ متریک کاربردی تکنولوژی انتخابی خود را با کد بخوانید و در گزارش نمایش دهید. استفاده از رابط گرافیکی مانند Kibana یا OpenSearch Dashboards برای مشاهده وضعیت سیستم مجاز است، اما به تنهایی کافی نیست.

برای این بخش باید یک برنامه بنویسید که متریک‌های لازم را از API، endpoint، driver یا روش رسمی تکنولوژی انتخابی جمع‌آوری کند. سپس نتایج را به صورت جدول و حداقل یک نمودار با استفاده از matplotlib در گزارش نمایش دهید.

متریک‌های پیشنهادی عبارت‌اند از:

- اندازه index یا داده ذخیره‌شده روی دیسک

- تعداد documentهای ذخیره‌شده

- زمان اجرای کوئری‌ها

- throughput یا تعداد کوئری در واحد زمان

- میزان مصرف CPU و حافظه

- P95 latency

- نرخ خطا در اجرای کوئری‌ها

کار با داده ذخیره‌شده

در این مرحله لازم است سه نوع جست‌وجوی زیر را روی دیتاست پیاده‌سازی و مقایسه کنید:

۱. Keyword Search: جست‌وجو بر اساس کلمات یا عبارات ورودی کاربر. در Elasticsearch می‌توانید از کوئری‌هایی مانند match_phrase و multi_match استفاده کنید.

۲. Contain Search: جست‌وجویی که بررسی کند یک کلمه، عبارت یا الگوی مشخص در عنوان یا چکیده مقاله وجود دارد یا نه. برای این بخش می‌توانید از روش‌هایی مانند wildcard، query_string یا طراحی analyzer مناسب استفاده کنید.

۳. Fuzzy Search: جست‌وجویی که در برابر غلط تایپی یا تفاوت‌های کوچک نوشتاری مقاوم باشد. این روش معمولاً سنگین‌تر است و باید اثر آن روی کارایی بررسی شود.

برای جلوگیری از ابهام، کوئری‌های این بخش به دو دسته تقسیم می‌شوند.

الف) کوئری‌های الزامی با روش مشخص

کوئری‌های زیر باید با روش مشخص‌شده اجرا شوند:

- Keyword Search: مقاله‌هایی را پیدا کنید که درباره query optimization هستند.
 - Keyword Search: مقاله‌هایی را پیدا کنید که درباره distributed database هستند.
 - Keyword Search: مقاله‌هایی را پیدا کنید که درباره learned indexes هستند و در دسته‌بندی cs.DB قرار دارند.
 - Contain Search: مقاله‌هایی را پیدا کنید که عبارت یا الگوی database systems در عنوان یا چکیده آن‌ها آمده است.
 - Contain Search: مقاله‌هایی را پیدا کنید که شامل indexing باشند ولی شامل blockchain نباشند.
 - Contain Search: مقاله‌هایی را پیدا کنید که شامل یکی از عبارت‌های query processing, storage engine یا transaction processing هستند.
 - Fuzzy Search: برای عبارت دارای غلط تایپی databse optimiztion, مقاله‌های مرتبط را پیدا کنید.
 - Fuzzy Search: برای عبارت دارای غلط تایپی transacton procesing, مقاله‌های مرتبط را پیدا کنید.
 - Aggregation/Analysis: برای یک query دلخواه، توزیع نتایج برگشتی را بر اساس سال و دسته‌بندی علمی گزارش کنید.
- برای هر کدام از موارد بالا باید latency, P95 latency, throughput, تعداد نتایج و متریک‌های مرتبط سیستم را گزارش کنید.
- (ب) کوثری‌های مقایسه‌ای
- برای سه عبارت زیر، هر چهار حالت keyword-only, contain-only, fuzzy-only و معماری ترکیبی پیشنهادی خود را اجرا و مقایسه کنید:
- learned indexes
 - query optimization
 - databse optimiztion
- برای این سه عبارت باید نتایج را از نظر تعداد نتایج، کیفیت نتایج، latency, P95 latency و throughput مقایسه کنید. برای بررسی کیفیت نتایج، حداقل ۱۰ نتیجه اول هر روش را به صورت دستی بررسی کرده و مشخص کنید چند نتیجه واقعا مرتبط بوده‌اند.

سناریوی جست‌وجوی ترکیبی

در این بخش باید یک معماری برای ترکیب سه روش جست‌وجوی بالا طراحی و پیاده‌سازی کنید. هدف این است که سیستم بتواند با هزینه مناسب، نتایج مرتبط را به کاربر برگرداند. یک معماری پیشنهادی این است که ابتدا keyword search و contain search به صورت موازی اجرا شوند. سپس نتایج این دو روش ترکیب و رتبه‌بندی شوند. اگر تعداد نتایج کافی نبود یا امتیاز نتایج از یک آستانه مشخص کمتر بود، fuzzy search به عنوان مرحله جایگزین اجرا شود. شما می‌توانید از این معماری استفاده کنید یا معماری پیشنهادی خود را ارائه دهید. در هر صورت، پیاده‌سازی معماری ترکیبی الزامی است و باید در گزارش توضیح دهید:

- چرا این معماری را انتخاب کرده‌اید؟

- در چه شرایطی fuzzy search اجرا می‌شود؟

- نتایج چند روش جست‌وجو چگونه ترکیب و رتبه‌بندی می‌شوند؟

- اگر یک مقاله در چند روش پیدا شود، امتیاز نهایی آن چگونه محاسبه می‌شود؟

- این معماری نسبت به اجرای همیشگی fuzzy search چه مزایا و معایبی دارد؟

مقایسه معماری ترکیبی با سه روش پایه باید حتماً روی سه کوئری مقایسه‌ای بخش قبل انجام شود. بنابراین، برای هر کدام از عبارت‌های query optimization, learned indexes و database optimiztion باید چهار خروجی زیر را گزارش کنید:

- خروجی keyword-only

- خروجی contain-only

- خروجی fuzzy-only

- خروجی معماری ترکیبی پیشنهادی

اجرای همزمان چند کلاینت

در این بخش منظور از چند کلاینت، اجرای چند برنامه، پردازش، نخ یا کاربر شبیه‌سازی شده است که به صورت همزمان به یک نمونه یا یک کلاستر از پایگاه داده متصل شده و کوئری اجرا می‌کنند.

حداقل ۱۰ ترکیب مختلف از اجرای همزمان نیازمندی‌ها را به واسطه چند کلاینت مختلف انجام دهید. تعداد کلاینت‌ها و کوئری‌های همزمان را به نحوی انتخاب کنید که کاهش کارایی به صورت قابل توجه مشاهده شود. سناریوهای الزامی عبارت‌اند از:

- اجرای keyword search با ۱، ۵ و ۱۰ کلاینت

- اجرای contain search با ۱، ۵ و ۱۰ کلاینت

- اجرای fuzzy search با ۱، ۵ و ۱۰ کلاینت

- اجرای معماری ترکیبی پیشنهادی با ۱۰ کلاینت

در هر سناریو باید میانگین latency، مقدار P95 latency، throughput، نرخ خطا و مصرف منابع را گزارش کنید. برای اجرای این بخش می‌توانید از ابزارهایی مانند Locust، k6، JMeter یا برنامه چندنخی/چندپردازه‌ای خودتان استفاده کنید.

بهینه‌سازی نحوه کار با داده

در رابطه با روش‌های بهینه‌سازی ذخیره‌سازی و جست‌وجو در تکنولوژی انتخابی خود تحقیق کنید و اثر آن‌ها را بررسی کنید. توجه کنید که در سیستم‌هایی مانند Elasticsearch داده‌ها از ابتدا در ساختاری مبتنی بر index ذخیره می‌شوند؛ بنابراین منظور از بهینه‌سازی، مقایسه یک طراحی اولیه ساده با یک طراحی بهتر برای mapping index، analyzer و نحوه اجرای کوئری‌ها است.

در صورت استفاده از Elasticsearch یا OpenSearch، موارد زیر می‌توانند بررسی شوند:

- طراحی بهتر mapping و انتخاب درست نوع فیلدها

- استفاده از multi-field برای فیلدهایی که هم جست‌وجوی متنی و هم فیلتر یا مرتب‌سازی نیاز دارند

- استفاده از filter context برای شرط‌هایی مانند سال و دسته‌بندی

- طراحی مناسب analyzer برای contain search

- مقایسه wildcard search با ngram بر روش مبتنی بر

- بررسی اثر تعداد shardها و replicaها

- بررسی اثر روش دسته‌ای ورود داده روی زمان ذخیره‌سازی

ابتدا کوئری‌های بخش «کار با داده ذخیره‌شده» را روی طراحی اولیه اجرا کنید. سپس با اعمال روش‌های بهینه‌سازی، همان کوئری‌ها را دوباره اجرا کنید و نتایج را مقایسه کنید. برای هر بهینه‌سازی، تاثیرات مثبت و منفی آن را بررسی کنید. برای مثال ممکن است استفاده از ngram باعث سریع‌تر شدن contain search شود، اما حجم index را افزایش دهد.

تحويل نهایی

تحويل نهایی پروژه باید شامل موارد زیر باشد:

- کد Python مربوط به پیش‌پردازش داده
- کد ذخیره‌سازی دسته‌ای داده در پایگاه داده
- فایل تنظیمات، mapping یا توضیح ساختار ذخیره‌سازی
- کد اجرای سه نوع جست‌وجو
- پیاده‌سازی معماری ترکیبی جست‌وجو
- کد جمع‌آوری متریک‌ها و رسم نمودارها با matplotlib
- کد اجرای آزمایش‌های چندکلاینتی
- فایل Docker Compose یا راهنمای کامل اجرای محیط پروژه
- گزارش نهایی شامل نتایج، متریک‌ها، مقایسه‌ها و تحلیل بهینه‌سازی‌ها